14 擴充資料結構

Part III Data Structures

本章定位

擴充資料結構要讓新查詢變快，同時把修改摘要的成本算清楚。本章用size建立動態rank/select，再證明何時能局部更新，並以精確max和閉區間端點推導interval tree只走一條路仍不漏答案。

前置知識

BST、size 欄位、區間與不變量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

本章路線

學習目標

- 以 subtree size 支援動態秩。
- 用固定步驟選擇可靠附加資訊。
- 設計 interval tree 回答重疊查詢。
- 14.1 Dynamic order statistics
- 14.2 How to augment a data structure
- 14.3 Interval trees

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14.1 Dynamic order statistics

第9章在靜態無序陣列找一次第k小需線性時間；若資料持續插入/刪除而反覆查rank/select，可在平衡BST保存子樹摘要。Order-statistic tree在red-black tree每node x加size，定義包含x本身的實節點數，NIL.size=0：

`size(x)=size(x.left)+size(x.right)+1`。

Rank採中序identity位置的1-index；相同key若各存一node，需用(key,identity)定全序，或明確以當前中序順序定義節點rank，不能只問重複值「唯一的rank」。本章數例鍵相異。

查詢的成本先是O(height+1)；只有底層維持第13章red-black性質，才可換成最壞O(log(n+1))。在普通BST上加size不會自動產生平衡。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Select的局部秩不變量

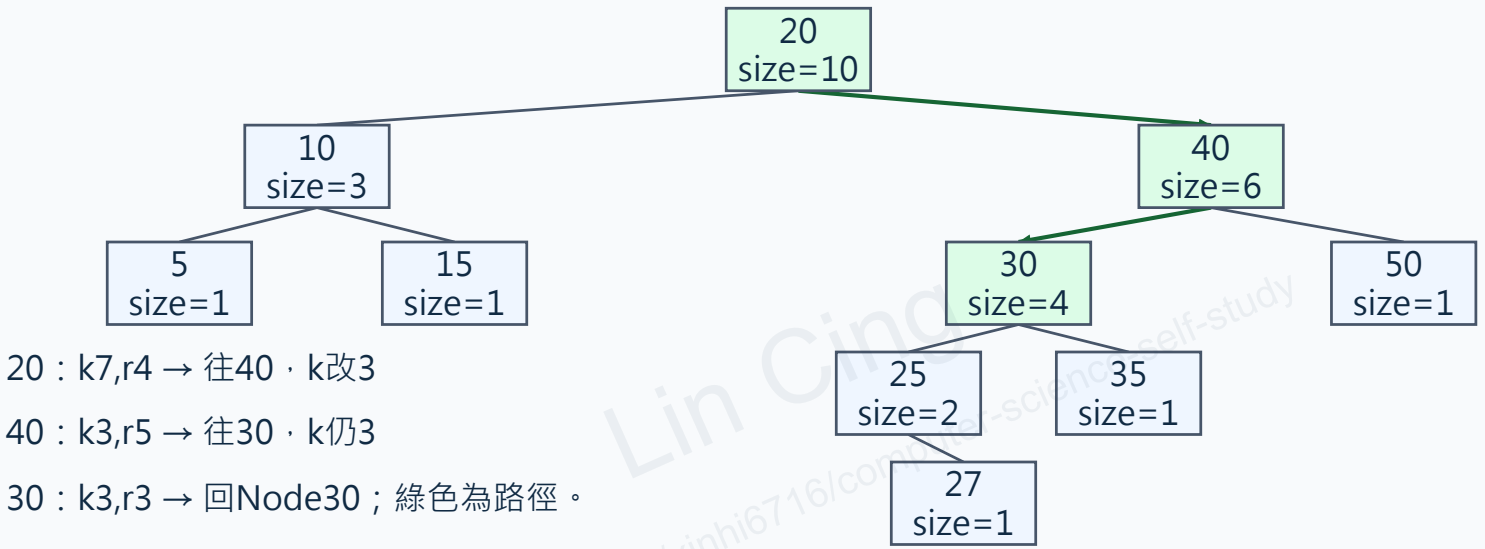
```
select(x,k):
    require 1<=k<=x.size
    while x!=NIL:
        r=x.left.size+1
        if k==r: return x
        if k<r: x=x.left
        else: k-=r; x=x.right
```

不變量：答案是目前x子樹的局部第k個identity。左子樹有r−1個節點、x佔第r位；往左不扣，往右扣掉完整左子樹及x共r筆。合法k條件保證所選側非空且新k合法，故不會遞迴進NIL；每次下降一層，終止時回正確node。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Select的局部秩不變量 （續）

Select第七小：Size與局部Rank同步



20：k7,r4 → 往40，k改3

40：k3,r5 → 往30，k仍3

30：k3,r3 → 回Node30；綠色為路徑。

第7小：root20的left.size=3、局部r=4，往40改k=3；40的left.size=4、r=5，往30仍k=3；30的left.size=2、r=3，回30。不能把全樹rank7直接拿到每層比較。

Rank每次只增加一塊尚未計入的前綴

```
rank(T,x):
    r=x.left.size+1; u=x
    while u!=T.root:
        if u==u.parent.right:
            r+=u.parent.left.size+1
        u=u.parent
    return r
```

不變量：r是原x在目前u子樹內的中序rank。若u是left child，parent及右側都在x之後，不新增前綴；若u是right child，parent.left全體和parent本身都在x之前，且不在先前u子樹內，恰加left.size+1，沒有重覆。到root即全樹rank。

同例rank27：初r=1；27是25.right，加0+1得2；25是30.left、30是40.left，各不加；40是20.right，加3+1得6。中序5,10,15,20,25,27,30,35,40,50的第6格確為27。

更新Size要跟著最終結構走

插入新葉size=1，只有祖先數量加1。可沿搜尋路徑計數，或接好後由parent往root重算；每個祖先O(1)，共O(height+1)。同key更新record不應誤把size加1。

刪除不能只對「被要求刪的z」一路減1後就結束：雙子情況後繼y從原位搬到z位置，y自身的子樹集合也改變。安全方法是完成第12章的transplant後，從受影響的最深祖先依序往上重算。

例刪20、搬25，先由25原父30往上：size30從4變3，size40從6變5，新root25須重算成1+3+5=9，不能保留25原size2。直接後繼分支可從搬上來的y開始；單子/leaf分支從z的原parent開始。

一般受影響node位於常數條祖先路徑的聯集，長O(height+1)。Red-black修復的旋轉還需同步更新size，下一節說明何時能只改兩個node。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

14.2 How to augment a data structure

設計擴充時要同時回答四件事：底層支援哪些基本操作？需要保存哪個摘要？每種修改如何維持摘要？新query如何用它剪枝或定位？只有寫一條f公式，不算完成更新與query的證明。

常用充分條件：每node的f可在O(1)時間由自身不隨recolor改變的key/data、左右child的f計算，NIL的f有明確常數基例。局部資料或child連結變動只可能影響該node與祖先，按後代先、祖先後重算。

Red-black插入的初次接葉、刪除的常數次transplant，各造成常數條O(log n)祖先路徑；每次rotation先修兩個轉軸node，再視需要往祖先傳播，最多O(log n)額外工作。第13章插入最多2次rotation、刪除最多3次，所以總仍O(log n)。

此一般論證只保證「每次rotation含傳播最多O(log n)」，不是所有f都能rotation後只重算兩格。若f依顏色，還須把recolor對依賴路徑的影響納入更新，不能直接用「顏色不改結構」忽略它。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Size、Max與一般摘要的差別

Left-rotate(x)，y=x.right升上來，β=y.left移給x。先完成pointer更新，再pull(x)、pull(y)，因新y依賴新x；反過來會讀到舊x摘要。

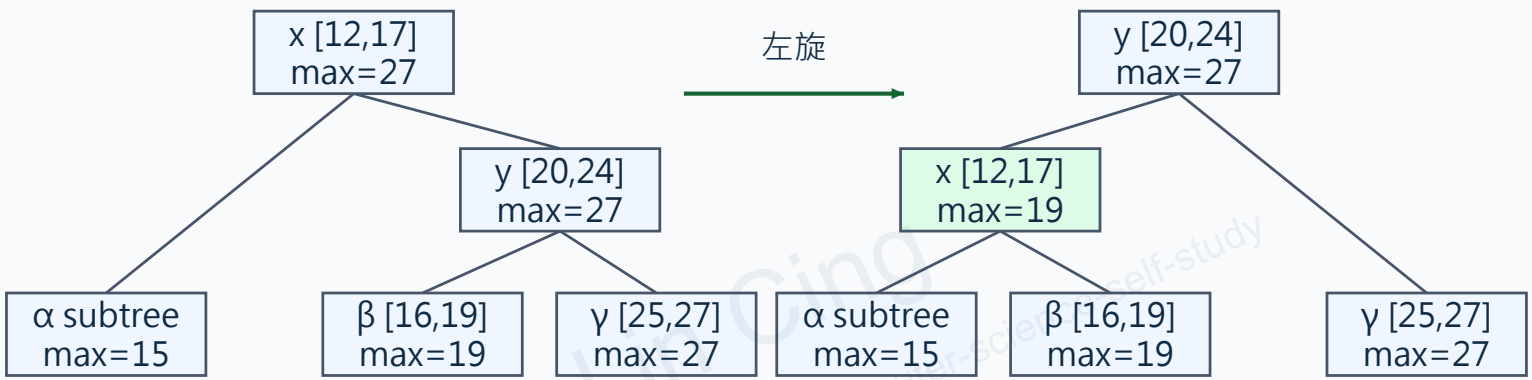
Size與「子樹所有interval的最大high」只依該子樹包含的內容，rotation後新y包含的內容恰為舊x整棵子樹，所以新y摘要等於舊x摘要，其他祖先的摘要不改。這兩種情況只需O(1)局部重算。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Size、Max與一般摘要的差別 （續）

局部Rotation：下移Node先更新摘要

原x=[12,17]、y=[20,24]；α.max15、β=[16,19]、γ=[25,27]。



先Pull新x：max(17,15,19)=19；再Pull新y：max(24,19,27)=27。

綠框：下移x的摘要改變；其他藍框摘要在旋轉前後相同。

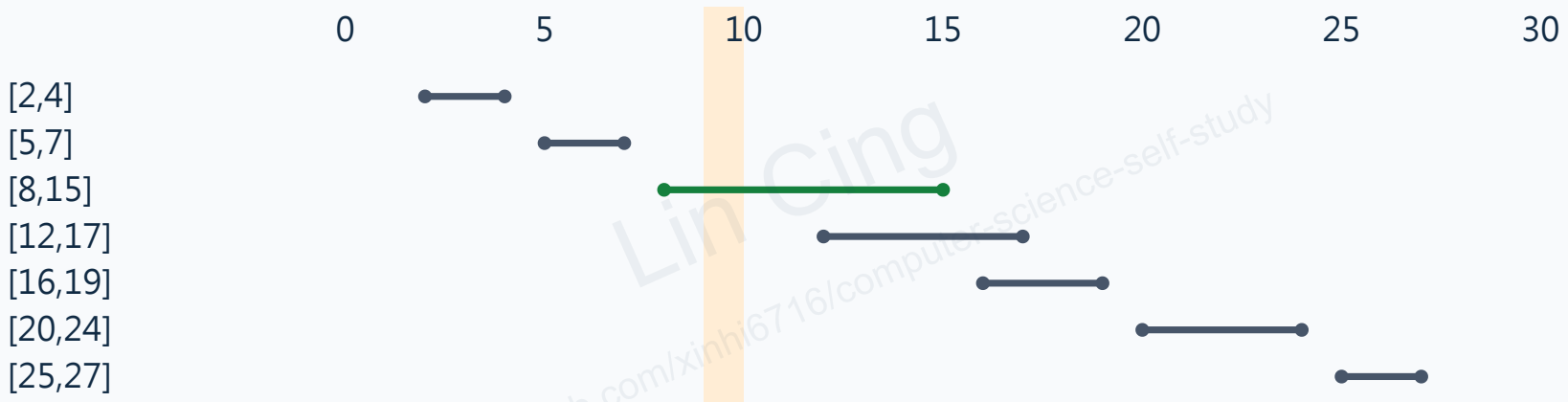
一般height摘要雖也由兩個child算出，rotation卻可能改變整棵局部子樹height，祖先仍須傳播。相反，若在每node直接存全樹rank，插新minimum會讓所有舊node的rank加1，需要Θ(n)次明寫；存depth也可能因一次rotation使大子樹全體depth改變。子樹size能在修改局部維護、查詢時沿路組合，是選它的原因。

14.3 Interval trees

本節interval是閉區間[l,h]，l≤h，允許退化為單點[l,l]。兩區間i、q重疊當且僅當i.low≤q.high且q.low≤i.high；等價於排除i完全在q左方（i.high<q.low）與右方（q.high<i.low）。

閉Interval與同一個Overlap Query

橘帶是閉Query [9,10]；實線兩端實心點表示閉端點。



綠：[8,15]重疊Query；灰：其餘不重疊。例：[4,5]與[5,7]在5重疊。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

14.3 Interval trees （續）

以low作BST主鍵；同low可加identity作tie-break，仍保留「left.low≤x.low≤right.low」。每node存max(x)=max(x.high,left.max,right.max)，NIL.max=−∞。這是**子樹**的最高high，不是該node的high，也不是最大的low。

本例root[12,17]的high17、max27；left[5,7]的max15來自其right[8,15]。只看root.high或當前interval，會漏掉伸得更遠的後代。

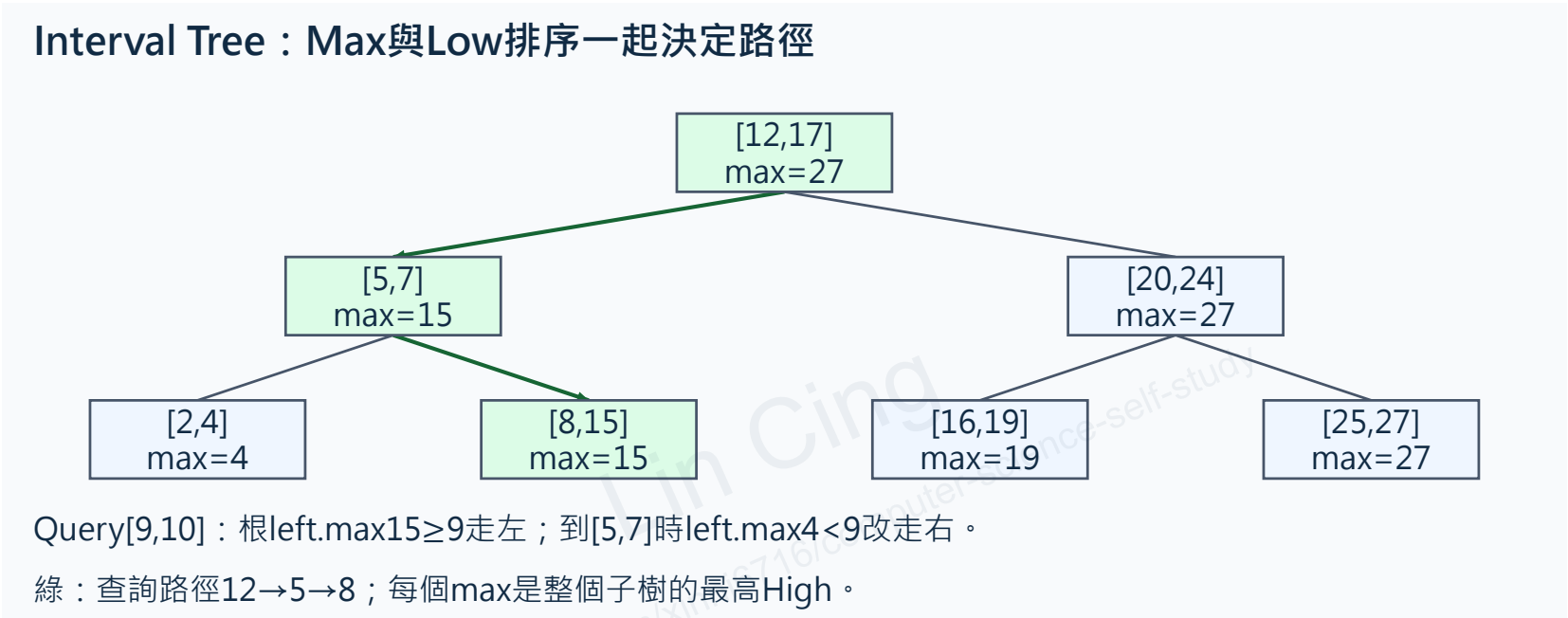
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一條路徑的Overlap search

```
interval-search(T,q):
    x=T.root
    while x!=NIL and not overlaps(x.interval,q):
        if x.left!=NIL and x.left.max>=q.low:
            x=x.left
        else:
            x=x.right
    return x
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

一條路徑的Overlap search （續）



Query[9,10]：root[12,17]不重疊，但left.max15≥9，往[5,7]；它也不重疊，left[2,4].max4<9，改往right[8,15]，重疊成功。

Query[18,18]：root不重疊且left.max15<18，往[20,24]；它不重疊、left.max19≥18，往[16,19]成功。

Query[28,29]依序[12,17]→[20,24]→[25,27]→NIL，回不存在。

往右的剪枝理由

維持不變量：若整棵樹存在重疊interval，當前x子樹也有一個。初始化root當然成立，當前x若重疊就回傳。若不重疊，要證明選方向不丟掉所有解。

當left=NIL，左側沒有候選；或left.max<q.low，任意左interval j都有j.high≤left.max<q.low，完全在query左方，所以整個left可排除。既然x也不重疊，若有答案只能在right，往右保持不變量。

這個證明使用所有left後代的high上界；「x.high<q.low」本身不足以推論left也全部太左。若只將例子的root改成[12,13]，query為[14,14]，root.high13<14，但left內的[8,15]仍重疊，不能因此剪掉left。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

往左時為何不必另回來找Right

若left.max≥q.low，令j是left中達到該max的interval。若left本有任何重疊，走左沒問題。若left完全沒有重疊，則j.high≥q.low卻不重疊，只能是j.low>q.high，不能是j.high<q.low。

對任意right後代k，BST主鍵序給`k.low≥x.low≥j.low>q.high`，所以right也全在query右方。加上x不重疊，此時x子樹本就沒有答案。故無論left有沒有答案，選left都保持「原本若有解，所選側必有解」。

這是選一條路徑仍正確的核心，不是宣稱只要left.max≥q.low，left一定有重疊。到NIL時所選子樹沒有解，由不變量反推全樹沒有解。每次下降一層，時間O(height+1)，red-black底層給O(log(n+1))。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

修改Interval後Max也必須向上修

插入/刪除後，沿受影響祖先pull max；rotation後先新x再新y。不能只在插入時把祖先max變大，刪掉最高high或縮短interval後，max也可能要下降；必須從自身high與兩child重新取max。

同例刪[25,27]，right[20,24]的max由27降24，root的max也由27降24。這說明摘要必須能下降；單是這次刪除，留下舊值的兩個node並非被讀取的left child，尚不是漏搜的具體反例。

真正的stale-left反例：先把原[8,15]改為[8,27]（low不變），其父[5,7]的max成27。再刪掉[8,27]，正確left subtree只剩[5,7]與[2,4]，max應降為7；假設錯留27。查[22,22]時，root[12,17]不重疊，卻因left.max27≥22錯往[5,7]，接著因[2,4].max4<22走到空right，回NIL；真正重疊的right[20,24]被漏掉。若max被低估，則可能直接剪掉真答案。要維持**精確**max，不是任意上界。

注意：[5,7]與[4,5]在閉端點5重疊；若應用採半開[5,7)、[4,5)，它們不重疊。半開非空interval須l<h，overlap改嚴格不等式，左側可行判斷也改left.max>q.low。不能只改圖上的括號而保留舊比較式。

找一個、找最左與列全部是不同Query

本節演算法只承諾回任一個重疊record，不保證low最小，也不列全部。若要列全部，可遞迴：left.max≥q.low才走left；檢查x；若x.low≤q.high且right.max≥q.low才走right。每個被訪nodeO(1)，最壞O(n)，不能直接把單一路徑O(log n)套到所有輸出。

另一個清楚可證的列舉法是重複overlap-search、暫時刪掉找到的record，記在列表；共找到k筆加最後一次失敗query，時間O((k+1)log(n+1))，再插回也同階，但會暫時修改樹，需符合呼叫者契約。輸出k筆本身至少Ω(k)，而要精確達O(log n+k)需另外分析或選更合適的表示，不能只寫「依輸出量」當證明。

Order-statistic與interval摘要可同時保存size、max，pull各O(1)，兩者的更新仍同階；但rank是依BST鍵(low,identity)排序，並不是high排序或重疊筆數，這些問題需要不同摘要/演算法。

Lin Cing https://github.com/xinhi6716/computer-science-self-study

資料來源

書目

章節來源

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009，Chapter 14。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

參考資料與署名

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study